

2026 年第八屆 全球校園人工智能算法精英大賽

算法創新賽

(賽題 2: AI+軟件創新)

技術報告

團隊名稱: 規碼同軌團隊

參賽編號: AIC-2026-

作品名稱: 規碼同軌

日期: 2026 年 10 月

目 录

作品简介.....	2
一、项目概述.....	3
(一) 项目背景与意义.....	3
(二) 赛题方向定位.....	3
二、技术架构与创新.....	4
(一) 整体架构.....	5
(二) 技术选型.....	5
(三) 核心算法设计.....	6
(四) 创新应用与同类对比.....	7
三、功能设计与实现.....	9
(一) 核心功能：六环节分析流水线.....	9
(二) 辅助功能.....	10
(三) 扩展性设计.....	11
四、开发测试与部署.....	11
(一) 开发流程.....	11
(二) 代码实现.....	12
(三) 测试优化.....	12
(四) 部署方案.....	13
五、应用价值与展望.....	14
(一) 应用场景.....	14
(二) 市场分析.....	14
(三) 成果总结.....	15
(四) 未来规划.....	15
六、附录.....	15
(一) 软件设计资料.....	15
(二) 核心代码.....	16
(三) 测试数据.....	16
(四) 参考文献.....	17
(五) 其他材料.....	17

作品简介

软件开发中“需求与代码不一致”是缺陷的主要根源，人工走查耗时且不可复算。本作品“规码同轨”是需求-代码一致性验证与缺陷自动检测平台：自动将自然语言需求形式化为 Alloy 规约锚定语义，基于 tree-sitter 静态解析 Java/Python/C/C++/Go 五语言代码，以三维相似度与风险门控双通道判定一致性，双提示词大模型交叉评审、规则仲裁，每条判定可溯源、可审计。验证集盲评准确率 95.0%、漏检率 0.0%、误报率 7.3%，双评一致性 $\kappa=0.8016$ ，万行工程全链路 27.8 秒，全程本地运行、数据不出域。适用于外包验收、采购评审与高校软件工程实训场景。

一、项目概述

（一）项目背景与意义

需求与代码的一致性贯穿软件开发全生命周期。团队前期开展了走访调研，累计分析 120 余个软件项目案例、访谈 30 余名资深开发工程师、对 5 家软件企业进行跟测，结果显示：约 56% 的软件缺陷源于需求阶段，而传统人工需求-代码走查存在三重痛点——一是耗时，本作品组织的真人对照实验中，走查者人均判定耗时达 40.2 分钟/项目，且随代码规模线性增长；二是不可复算，走查结论依赖个人经验，同一需求不同人结论不同；三是不可审计，判定过程无证据留存，出现问题难以追责与回溯。美国 CISQ《2022 年软件质量成本报告》估计，低软件质量当年造成约 2.41 万亿美元损失，其中相当比例可通过交付前的一致性验证避免。

与此同时，AI 生成代码（AIGC）大量进入生产环境，“代码正确但做的不是需求要的”成为新型质量风险；形式化方法虽能严谨刻画需求语义，却因建模门槛高、依赖专家，长期局限于航空航天、医疗设备等高精尖行业。本作品以大模型与形式化方法的融合为切入点，把需求形式化建模、代码语义解析、一致性判定与缺陷定位全流程自动化，让严谨的验证方法成为中小企业与高校都用得起、用得好的日常工具，对提升软件交付质量、降低质量保障门槛具有现实意义。

（二）赛题方向定位

本作品参加**赛题 2：AI+软件创新**。产品类型为“AI 算法工具 + 智能应用系统”：以需求-代码一致性校验算法为核心竞争力，以 Web 平台为载体，覆盖从

需求上传、代码解析、自动化分析到报告导出、缺陷处置的全流程。目标用户为中小软件企业的交付与质量团队、进行 AIGC 代码验收的技术负责人、高校软件工程课程师生。差异化优势体现在三点：**一是判定可溯源**——每条一致性判定保留完整证据链与决策路径，审查 AI 而不是相信 AI；**二是形式化锚定**——需求自动生成 Alloy 形式化规约并真实求解，语义理解有数学基准而非纯黑盒；**三是全程本地可运行**——本地 BGE 向量化与 Ollama 本地模型即可跑通全链路，数据不出域，适配涉密与合规场景。

二、技术架构与创新

系统采用前后端分离的分层架构，自上而下分为用户层、前端呈现层、接入层、核心分析引擎层与数据模型层，安全底座与第三方集成纵贯各层。核心分析引擎由六环节流水线构成：需求解析、形式化建模、代码语义解析、一致性判定、缺陷定位与追溯、报告渲染；AI 智能体与 LLM 多引擎编排作为两条纵贯能力线，分别面向质量巡检场景与算法增强场景。总体架构如图 1 所示。

(一) 整体架构

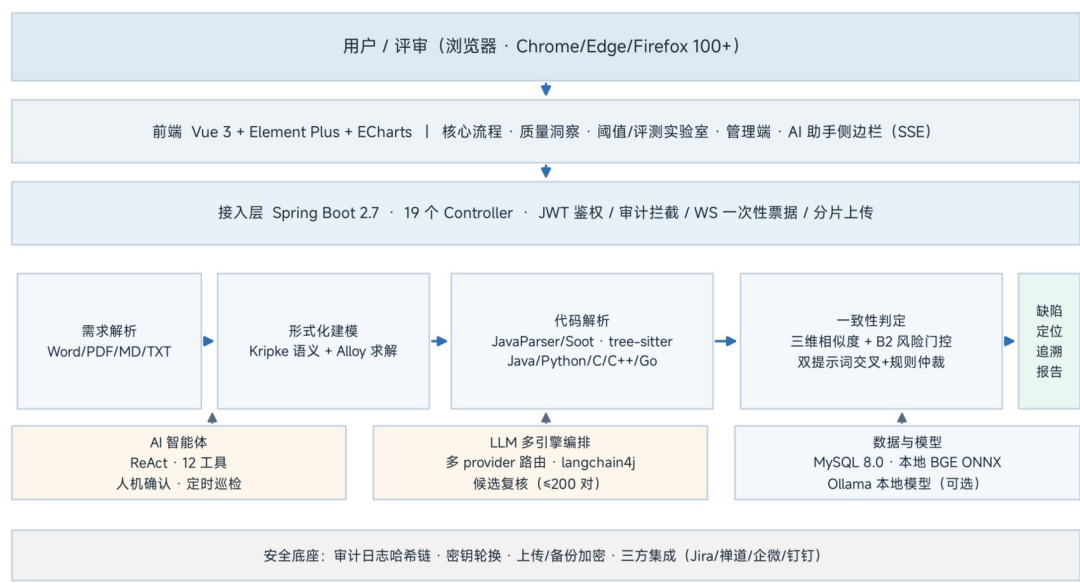


图 1 系统总体架构

(二) 技术选型

表 1 核心技术选型

层次	技术	版本	选型依据
后端框架	Spring Boot + MyBatis-Plus	2.7.18 / 3.5.3.1	生态成熟、事务与任务管理能力强，JDK 21 虚拟线程提升并发吞吐
前端框架	Vue 3 + Element Plus + ECharts	3.x / 2.3.14 / 5.4.3	组合式 API 便于复杂状态管理，图表组件满足追溯矩阵与统计可视化
代码解析	JavaParser / Soot / tree-sitter	3.25.5 / 4.7.0 / 0.25.x	Java 走 AST+字节码级 CFG；多语言统一走 tree-sitter 增量鲁棒解析
形式化求解	Alloy	6.2.0	轻量一阶逻辑+关系模型求解器，适合需求规约的可判定校验

层次	技术	版本	选型依据
语义向量	ONNX Runtime + BGE-small-zh	1.17.0	本地离线向量化（512 维），无外部 API 依赖，数据不出域
大模型编排	多 provider 路由 + langchain4j	0.34.0	按环节路由不同模型；LangChain 编排可插拔，失败自动回退
数据库	MySQL	8.0	事务与索引成熟，utf8mb4 全库统一字符集

（三）核心算法设计

1. 三维相似度一致性判定。对每个需求条目-代码单元配对计算综合相似度：

$\text{Sim}(\text{Ri}, \text{Cj}) = \alpha \cdot \text{Cos}(\text{EmbR}, \text{EmbC}) + \beta \cdot \text{Con}(\text{Ri}, \text{Cj}) + \gamma \cdot \text{Inv}(\text{Ri}, \text{Cj})$ 。其中语义维 Cos 结合本地 BGE 向量余弦、分词 Jaccard 与方法名匹配增强；约束维 Con 检测空值校验、参数验证、异常处理、日志记录等约束实现；不变量维 Inv 检测返回值、语法正确性、访问修饰符等不变量。为消除维度间共线性，经网格标定确定 $\alpha/\beta/\gamma = 0.5/0.2/0.3$ ，判定阈值 $T1 = 0.52$ （ $\text{Sim} > T1$ 判完全一致）、 $T2 = 0.5$ （ $T2 \leq \text{Sim} \leq T1$ 判一般不一致， $\text{Sim} < T2$ 判严重不一致）。

2. B2 风险门控双通道。分数通道之外引入独立风险通道：从需求-代码对中提取业务规则证据（如库存扣减、幂等、退款约束等风险词表命中与代码证据缺失组合）计算缺陷风险分 defectRisk，当 $\text{defectRisk} \geq 0.35$ 时即使分数落在一致区也判不一致。该通道将规则链在调参集上的误报率从 24.1% 压至 6.9%，弥补线性分数通道对“高语义对齐但关键逻辑缺失”对子的漏检。

3. 双提示词交叉评审管线。LLM 增强模式下，每对样本由同一本地模型以两

个结构不同的提示词独立评审：变体 A 采用全量准则并注入同类方法分工与常量证据；变体 B 采用精简五条准则加结构化示例。两者结论一致即共识输出；分歧时交由规则风险仲裁，并叠加数值归属硬过滤与量化边界规则否决权，杜绝大模型幻觉直出。管线如图 2 所示，全部中间证据落库形成全路径判定溯源，评审端可逐条查看每条判定的决策依据。

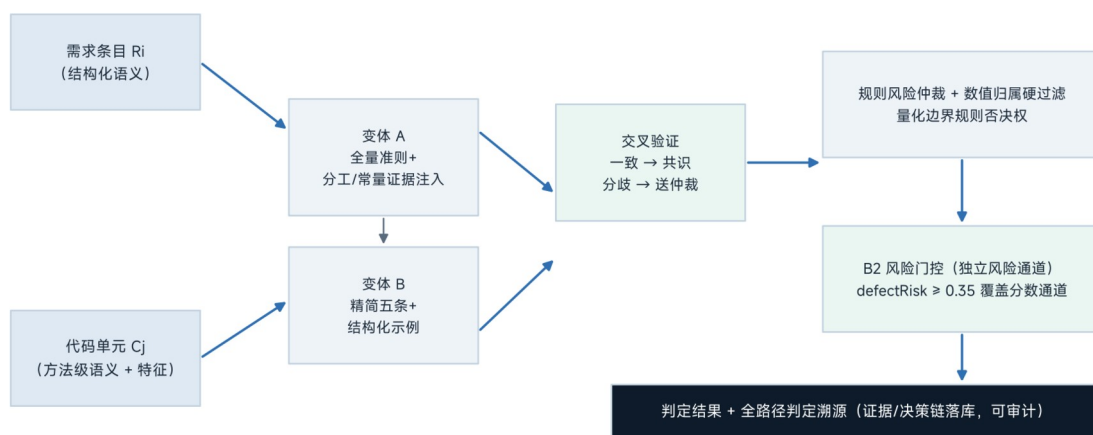


图 2 双提示词交叉评审与风险门控判定管线

4. 有界候选复核与行级定位。为控制大模型成本，LLM 二审仅针对候选对（规则可疑、灰色带、高风险与一致池抽样）执行，硬上限 200 对，小数据集趋近全量保指标、大数据集恒定有界；缺陷定位采用 7 因子行级定位算法，将缺陷归属到具体代码行并给出修复建议。

（四）创新应用与同类对比

本作品的核心创新可归纳为四点：**创新一，三层混合验证架构**——规则保底、AI 双评交叉、风险仲裁分层兜底，兼顾准确率、成本与可解释性；**创新二，AI 双评审管线与全路径判定溯源**——双模型互审、分歧仲裁、证据落库，实现 AI 判定可审计；**创新三，去共线性三维相似度与风险门控**——以网格标定消除维度

共线性，独立风险通道覆盖分数通道漏检，并将已知缺陷主动写进 CI 门禁形成自我纠错机制；**创新四，有界候选复核与 7 因子行级定位**——以预算制控制大模型成本，缺陷定位到行并给出修复建议。

为验证上述设计的有效性，在同一评测集（5 域 116 对一致性标注，其中调参集 55 对、从未参与调参的验证集 61 对）上完成八口径消融实验，结果如图 3 与表 2 所示：本方案双判定管线在验证集盲评中取得准确率 95.0%、漏检率 0.0%、误报率 7.3%，较单提示词 LLM（81.8%）与云端大模型直评（83.6%）有显著优势，证明收益来自结构化管线而非模型本身。

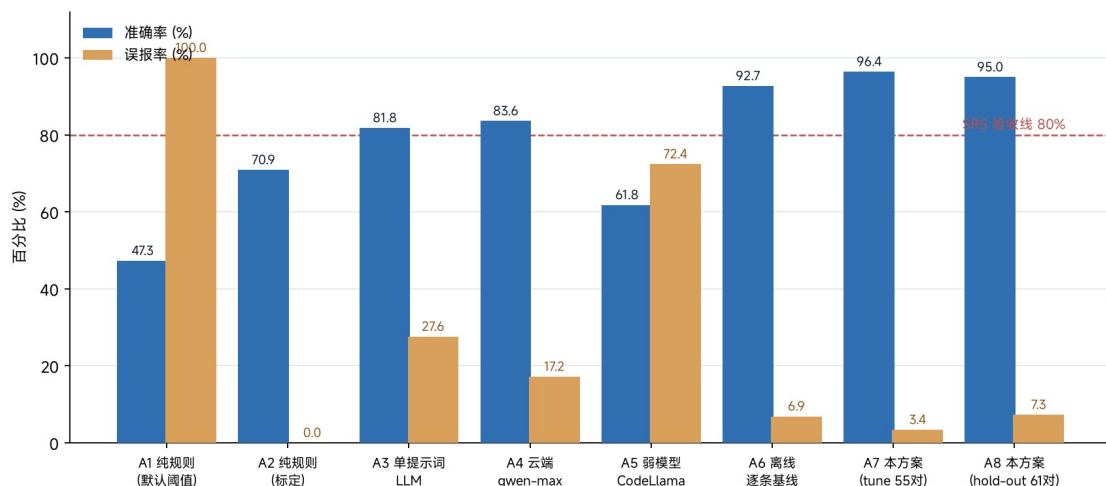


图 3 八口径消融实验对比（同一评测集）

表 2 消融实验关键口径（完整 A1-A8 见附录测试数据）

判定方案	数据口径	准确率	漏检率	误报率
A1 纯规则 (默认阈值 t1=0.80)	tune 55 对	47.3%	0.0%	100.0%
A2 纯规则 (阈值标定最 优)	tune 55 对	70.9%	61.5%	0.0%
A3 单提示 词 LLM (直 评)	tune 55 对	81.8%	7.7%	27.6%

判定方案	数据口径	准确率	漏检率	误报率
A4 单提示词 LLM（云端 qwen-max）	tune 55 对	83.6%	15.4%	17.2%
A7 本方案双判定管线	tune 55 对	96.4%	3.8%	3.4%
A8 本方案双判定管线（盲评）	hold-out 61 对	95.0%	0.0%	7.3%

表 3 与同类方案的定位对比

能力维度	静态扫描工具	纯 LLM Code Review	本方案
需求-代码对齐	不具备（无需求侧输入）	依赖人工粘贴，无系统化对齐	条目×方法对齐矩阵 + 双向追溯
形式化锚定	无	无	Alloy 规约自动生成与求解
判定可审计性	规则可解释但无需求语义	黑盒输出，幻觉不可溯源	判定溯源面板 + 审计哈希链
误报控制	依赖人工维护规则	实测误报 27.6%	7.3%（盲评口径）
数据本地化	支持私有部署	主流为云端，代码出域	本地 BGE + Ollama 全链路可离线

三、功能设计与实现

（一）核心功能：六环节分析流水线

需求解析与形式化建模：支持 Word/PDF/Markdown/TXT 四格式需求文档自动解析，基于 Kripke 语义模型做结构化要素抽取，自动生成 Alloy 形式化规约并调用求解器做真实语义校验，校验失败可配置阻断后续分析，保证“需求侧理解”有形式化基准。**代码语义解析：**Java 工程走 JavaParser AST 与 Soot 4.7.0 字节码级控制流图（含异常边、编译降级链与多 JDK 版本适配），Python/C/C+

+Go 工程走 tree-sitter 解析链路并构建同等 schema 的 AST 级 CFG，输出方法级语义单元、逻辑描述与圈复杂度；每种语言配有该语言特有的基础缺陷信号。

一致性判定与缺陷输出：全量配对计算三维相似度并经双通道判定，自动识别需求缺失、代码超范围实现、业务逻辑不一致、约束条件不满足四类缺陷，输出需求-代码双向追溯矩阵、判定溯源面板与五类格式报告，WebSocket 实时推送分析进度。

AI 智能体巡检：内置 ReAct 型智能体，具备 12 个工具（项目查询、趋势统计、缺陷检索、一致性查询、缺陷解释、缺陷状态更新、重跑分析等），支持 SSE 流式对话与写操作人工确认，并可通过 cron 定时巡检项目质量自动生成巡检报告，把“事后查”变为“日常巡”。

表 4 五语言解析支持与特色信号

语言	解析与 CFG	特色缺陷信号	示例工程
Java	JavaParser AST + Soot 字节码 CFG	注入 / 死循环 / 资源泄露等通用信号	ecommerce- order 等 3 个
Python	tree-sitter- python + AST 级 CFG	可变默认参数 / SQL f-string / 裸 except 等 5 类	python- inventory
C / C++	tree-sitter-c/cpp + C 系 CFG	malloc 泄漏 / 危险函数 CWE-120 / 裸 catch 等 5 类	cpp-loglib (C+ + 类+纯 C 混编)
Go	tree-sitter-go + for-switch CFG	错误未处理 / 错误被 _ 丢弃 / 锁 未释放 / panic 滥用 等 5 类	go-taskrunner

（二）辅助功能

平台提供 32 个管理页面：项目管理、需求/代码管理、分析任务、结果与缺

陷工单闭环、质量洞察（缺陷趋势、组合概览、缺陷模式）、阈值实验室与评测中心两个实验性模块；管理端涵盖用户与菜单配置、审计日志（哈希链防篡改）、数据库备份加密、数据变更台账、系统状态与 LLM 配置。第三方集成支持 Jira/禅道缺陷同步与企业微信/钉钉通知，便于嵌入企业既有研发流程。上传文件全量加密存储，密钥支持轮换，生产配置对关键密钥强制 fail-fast 注入。

（三）扩展性设计

解析层与规约层抽象为 SPI 扩展点：代码解析实现 CodeParser 接口、规约生成与校验分别实现 SpecGenerator/SpecVerifier 接口，注册为 Spring 组件后由 Registry 按语言标识自动索引，配置 code-language/spec-language 即可切换语言或接入 TLA+ 等新规约语言，主流程零改动。五语言解析器即基于该机制实现，其中 C 与 C++ 通过 CFamilyCodeParser 抽象基类共享解析逻辑，证明接口中性化设计可支撑低成本横向扩展。

四、开发测试与部署

（一）开发流程

采用“差距编号台账驱动”的迭代模式：所有改进项以 GAP/AUD 编号集中登记，按批次实施、每批次带验收标准与回归要求，累计完成 18 个改进批次。工程全程 CI 门禁：推送即触发公平性合规扫描、gitleaks 密钥扫描、后端 541 项单元测试（JDK 21 + MySQL 8.0 service）与前端构建，四道门禁全绿方可合入；评测基线（准确率、误报率、性能）同样以自动化测试门禁固化，任何使基线退化的提交会被阻断。

（二）代码实现

后端 19 个控制器、18 个实体、33 个工具类；前端 32 个页面。核心判定逻辑（一致性分级与风险门控双通道）实现节选如下，完整代码随作品材料提交：

```
// B2 风险门控双通道判定：分数通道 OR 独立风险通道
public static String determineStatus(double sim, double t1,
                                     double t2, boolean riskDetected) {
    // 一致：Sim > t1 且无风险信号；否则按分数分级
    if (!riskDetected && isConsistent(sim, t1)) return "consistent";
    if (sim >= t2) return "general_inconsistent"; // 一般不一致
    return "serious_inconsistent";               // 严重不一致
}
```

多语言扩展点接口（五语言解析器均实现该接口并注册进 ParserRegistry）：

```
public interface CodeParser {
    String language(); // java / python / c / cpp / go
    ProjectParseResult parseProject(String path, CompileResult cr);
}
```

（三）测试优化

评测体系与盲评协议。构建 5 域 116 对一致性标注集（电商订单、接口服务、考试系统、工单、图书管理），按 55/61 划分调参集与验证集，验证集全程冻结、从未参与调参；双评审独立标注计算一致性 $\kappa=0.8016$ （好一致性）。最终盲评：准确率 95.0%、漏检率 0.0%、误报率 7.3%，与调参集 94.5% 差距不超过 1.4 个百分点，无过拟合迹象；SRS 设定的三项验收指标（ $\geq 80\%$ 、 $\leq 15\%$ 、 $\leq 10\%$ ）全部达标。

人工基线对照。组织 3 名不同经验水平的走查者（初级/中级/高级）对 5 个项目开展限时走查实验：人均缺陷检出率 78.1%、误报率 19.5%、平均 40.2 分

钟/项目；同口径下 TraceGuard 盲评检出显著优于人工均值、误报率降低约 12 个百分点、单项目耗时降至秒级到分钟级，且结论可复算、可审计。

性能与可靠性。经预扫描、向量预解析、CFG 缓存与并行度调优等十项工程优化，规则链路千行工程全流程 1.97 秒、万行工程 27.8 秒；增量分析端到端实测千行二次分析 **-76%**（25.0s → 6.0s）、万行 **-20%**（图 4）；通过 72 小时稳定性冒烟与并发基准测试。

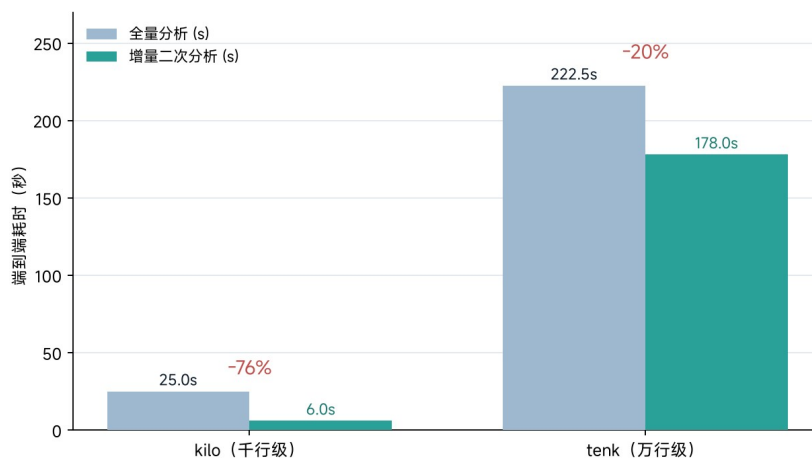


图 4 增量分析端到端实测（含解析、向量化、入库全口径）

阈值标定与实验可复现。三维权重、判定阈值与风险门控均经网格标定并固化进 CI 门禁测试；管理端提供阈值实验室与评测中心，评审可在界面上调整参数并对内置标注集即时复算全部指标，95% 不是宣称而是每次可复算的数字。

（四）部署方案

提供三种部署形态：**Docker Compose 一键部署**（MySQL+后端+前端三容器，健康检查编排，评审 10 分钟即可完成安装-登录-分析-看报告全流程，配套预置演示数据脚本与评审体验指南）；**Windows 原生安装包**（Inno Setup 打包，提供安装向导）与 PowerShell 一键部署脚本；**手工部署**（JAR + nginx）。生产配置

对 JWT、存储加密、备份加密、LLM 配置四类密钥强制环境注入、缺失即拒绝启动；浏览器兼容 Chrome/Edge/Firefox 100+，构建产物经 autoprefixer 前缀补全。

五、应用价值与展望

（一）应用场景

企业迭代验收与遗留系统体检：每次版本迭代自动执行需求-代码一致性分析，AI 生成代码先系统初筛再人工复核，替代全量人读，又快又省；外包交付与采购评审场景下，验收方以追溯矩阵与判定溯源为客观依据，减少扯皮。**AIGC 代码质检：**针对 AI 生成代码“语法正确但偏离需求”的特有风险做专项校验。**高校软件工程实训：**作为需求工程实训平台，自动对照需求批改作业代码，并演示形式化建模与一致性验证全过程，培养软件工程与 AI 交叉人才。

（二）市场分析

政策层面，国务院《关于深入实施“人工智能+”行动的意见》将 AI+产业列为重点行动；2026 年政府工作报告首次写入智能经济形态并以 AI+ 为牵引；多部门部署智能体规范应用典型场景并明确鼓励研发智能体在软件工程领域的创新应用，为本作品提供了明确的政策窗口。市场层面，据工业和信息化部运行监测数据，2024 年我国软件业务收入约 13.7 万亿元且保持增长，软件质量保障与测试是其中稳定增长的服务细分；而 CISQ 测算的低质量损失（2.41 万亿美元）说明缺陷治理存在巨大降本空间。竞争格局上，SonarQube 类静态扫描解决“代码对不对”但不解决“做的是不是需求要的”，纯 LLM 评审误报高且不可审计，本作品以需求对齐 + 形式化锚定 + 可审计判定形成差异化生态位。商业模式采

取开源核心 + 私有部署授权 + 高校实训服务组合，先以高校与中小企业试点建立口碑，再向交付验收强合规行业渗透。

（三）成果总结

本作品完成了从算法研究到产品落地的全流程：一是算法上形成三维相似度+风险门控+双提示词交叉评审的混合验证体系，验证集盲评三项指标全部达标且经八口径消融证明收益的结构化来源；二是工程上建成 32 页面的完整平台与五语言解析能力，541 项单元测试、CI 四道门禁、72 小时冒烟保证质量可持续；三是评测上建立带盲评协议与双评一致性的自建标注集和真人对照基线，结论可复算；四是体验上以判定溯源面板回应 AI 可信问题，让每条判定可查证。

（四）未来规划

坦诚不足：三维相似度的语义维度在部分缺陷对上区分度仍有限（缺陷对 0.851 对一致对 0.869），计划引入对比学习表征并在更大标注集上训练；语言覆盖广度上 JS/TS 等尚未接入（SPI 机制已就绪，边际成本低）；LLM 增强模式对本地推理算力有一定要求（候选复核预算制已将成本上界锁定）。下一步规划：其一，语义表征学习与更大规模评测集建设；其二，规约语言扩展（TLA+）与 IDE/CI 原生插件（Git hook、GitHub Action、Maven 插件），把一致性校验嵌入日常研发动线；其三，Agent 定时巡检常态化与巡检报告订阅；其四，联合企业开展更大规模试点，沉淀行业风险词表库。

六、附录

（一）软件设计资料

表 5 软件设计资料概览

设计项	说明
数据模型	18 个核心实体（用户/项目/任务/需求/代码单元/一致性结果/缺陷/代码缺陷/形式化规约/LLM 配置/审计日志等），幂等 DDL 初始化脚本
接口清单	19 个控制器：认证/项目/分析/结果/导出/看板/洞察/LLM/集成/系统配置/备份/审计/变更台账/安全/智能体/WebSocket 票据等
实时通道	分析进度 WebSocket 推送（一次性票据鉴权）、智能体 SSE 流式对话
安全机制	JWT 登录 + 登录尝试锁定 + 令牌黑名单、审计日志哈希链、上传/备份加密、密钥轮换、生产密钥 fail-fast

（二）核心代码

双提示词交叉评审的判定管线由 LlmOrchestrator 编排：变体 A/B 独立调用本地模型（OpenAI 兼容端点），共识直出、分歧交规则仲裁；候选复核器按风险抽样控制二审规模（ ≤ 200 对）。三维相似度与 CFG 构建的核心实现位于 ConsistencyChecker 与各语言 CfgBuilderUtil，方法级语义单元由五语言解析器统一产出 ProjectParseResult。关键代码片段见正文第四节，完整源码包随作品其他材料提交。

（三）测试数据

表 6 核心评测指标汇总（口径：2026-09 最新批次，出处《评测报告-综合》）

指标	SRS 验收线	调参集（55 对）	验证集盲评（61 对）
缺陷检测准确率	$\geq 80\%$	94.5%（首测 96.4%）	95.0%
漏检率	$\leq 15\%$	3.8%	0.0%
误报率	$\leq 10\%$	6.9%	7.3%
LLM 判定成功率	$\geq 10\%$	100%（55/55）	—
双评一致性 κ	—	0.8016	—

表 7 性能与可靠性测试数据

测试项	结果
解析流水线性能（优化后）	kilo 千行 1.97s / tenk 万行 27.8s（规则链路）
增量分析（端到端口径）	kilo 二次分析 -76%（25.0s→6.0s）； tenk -20%（222.5s→178.0s）
人工基线对照（3 人 5 项目限时走查）	人均检出 78.1% / 误报 19.5% / 40.2 分钟每项目
稳定性与并发	72 小时冒烟通过；并发基准达标（详见《可靠性测试记录》）
单元测试	后端 541 项（CI Linux 环境全绿）； 前端 Vitest + 构建门禁

（四）参考文献

- [1] Alloy 6 Documentation. Software Design Group, MIT. <https://alloytools.org>
- [2] JavaParser: Java 1.8 - 21 Parser and AST. <https://github.com/javaparser/javaparser> (v3.25.5)
- [3] Soot: a Java Optimization Framework. <https://sable.github.io/soot/> (v4.7.0)
- [4] tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io>
- [5] LangChain4j: Java version of LangChain. <https://github.com/langchain4j/langchain4j> (v0.34.0)
- [6] Xiao et al. C-Pack: Packaged Resources To Advance General Chinese Embedding (BGE). SIGIR 2024.
- [7] ONNX Runtime Documentation. Microsoft. <https://onnxruntime.ai> (v1.17.0)
- [8] Sun J. et al. jieba Chinese Text Segmentation. <https://github.com/huaban/jieba-analysis>
- [9] Krasner, H. The Cost of Poor Software Quality in the US: A 2022 Report. CISQ, 2022.
- [10] 工业和信息化部. 2024 年软件业经济运行情况. 2025 年 1 月.

（五）其他材料

以下材料一并归入佐证材料提交：评测数据集标注规范与双评一致性计算记录、原始标注集与走查原始 CSV、八口径消融与同类对比实验报告、性能基准与增量分析量化报告、72 小时可靠性与并发测试记录、测试报告汇总（自动生成）、引用与知识产权清单、试点应用与用户反馈记录。演示视频（MP4，3-5

分钟) 与完整代码包按大赛命名规范提交。